K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

K.R. Mangalam University, Gurugram, Haryana

ASSIGNMENT -2

Course Code & Title	**Analysis & Design of Algorithms, ENCS202**
Programme & School	B.Tech Computer Science & Engineering, SOET
Faculty Name	Dr. Vandna Batra
Type of Assignment	Continuous In-Class Evaluation
Weightage/Marks	5
Submission Mode	Handwritten (Classroom)

Name: Badal
Roll no: 2501351020

Q1.

Vaidehi 7/4/26

Questions

Apply **Merge Sort** to sort the following array:

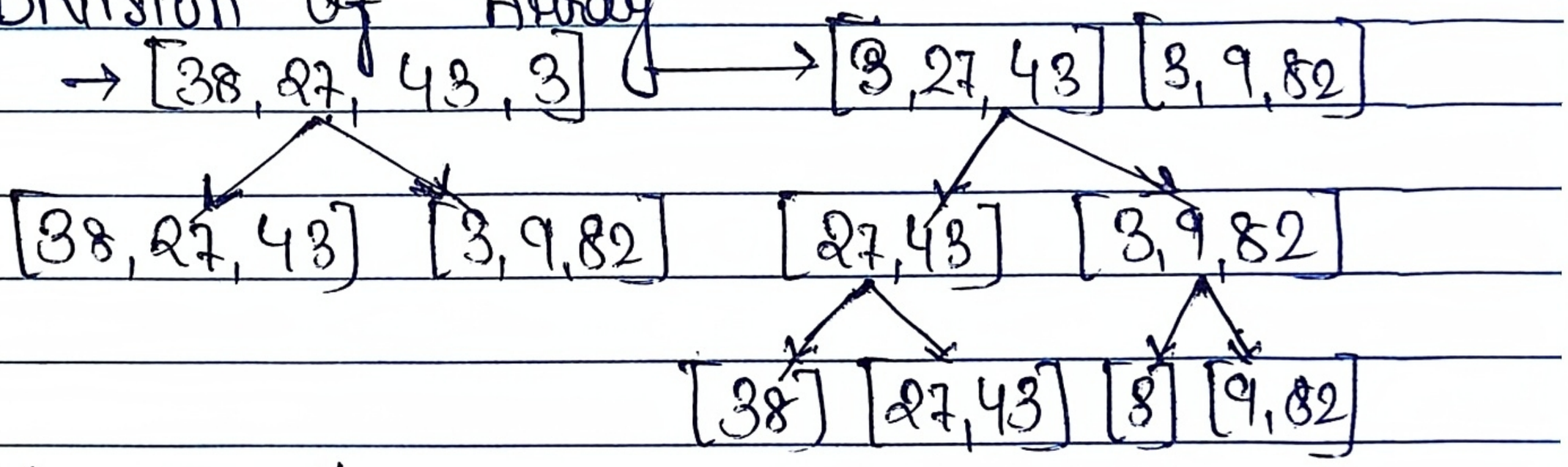
38, 27, 43, 3, 9, 82

i. Show the division of arrays

ii. Show the merging steps

iii. Write the recurrence relation for **Merge Sort** time complexity and solve it using the **Master Theorem**.

(i) Division of Array



(ii) Merging steps

step 1: Merge single element

[27] + [43] → [27, 43]

[27, 43] → [9, 82]

step 2: Merge further

[38] + [27, 43] → [27, 43] → [27, 38, 43] [3] → [9, 82]

step 3: final merge

[27, 38, 43] + [3, 9, 82] → Compare and [3, 9, 27, 38, 42, 82]

→ final sorted Array [3, 9, 27, 38, 43, 82]

(iii) Recurrence Relation & Time Complexity

→ $T(n) = 2T(n/2) + n$

→ $2T(n/2)$ → division into two subproblems

n → merging step

→ Using Master Theorem: $T(n) = aT(n/b) + f(n)$

Here: $a = 2$, $b = 2$ $f(n) = n$

Comparing: $n^{\log_b a} (a = n^{\log_2(2)} = n$

→ $f(n) \sim \Theta(n^{\log_b a})$ Case 2 of Master Theorem applies

Final Time Complexity

$T(n) = \Theta(n \log n)$ → $\boxed{T(n) = \Theta(n \log n)}$

→ Merge sort uses Divide & Conquer

→ Time Complexity = $O(n \log n)$

→ Stable and efficient for large data.

2. Apply Quick Sort to the array

1, 2, 3, 4, 5

by choosing the **first element as pivot** at every step.

i. Show the partition at each step

ii. Explain why this represents the **worst case**

iii. State the time complexity

(i) Partition at Each Step

Initial Array [1, 2, 3, 4, 5]

Step 1: Pivot = 1
left = []
Right = [2, 3, 4, 5]

Result:
[] + [2, 3, 4, 5]

Step 2: Subarray: [3, 4, 5]
Pivot = 3
left = []
Right = [4, 5]

Result: [] 3 [4, 5]

Step 4: Subarray: [4, 5]
Pivot = 4
left = [], Right = [5]

Result : [] 4 [5]

Step 5: Subarray : [5] (Already Sorted)

Final Sorted Array [1,2,3,4,5]

(ii) Why this is Worst Case

At every step:
- Pivot divides array into :
  left = empty
  Right = (n-1)
+ O(n)

This happens because:
- Array is already sorted.
- Pivot is always smallest element.

(iii) Time Complexity

$T(n) = T(n-1) + n$

Solving

$= n + (n-1) + (n-2) + \ldots + 1$

$= n(n+1)/2$

Final Time Complexity

$\boxed{T(n) = O(n^2)}$

3. Assume that we have a knapsack with max weight capacity, W = 16. Our objective is to fill the knapsack with items such that the benefit (value or profit) is maximum. Consider the following items and their associated weight and value

ITEM	WEIGHT	VALUE
i1	6	6
i2	10	2
i3	3	1
i4	5	8
i5	1	3
i6	3	5

W = 16

→ Given Data.

Step1: Calculate Value / Weight Ratio

Item	Value / Weight
I1	6/6 = 1
I2	2/10 = 0.2
I3	1/3 = 0.33
I4	8/5 = 1.6
I5	8/1 = 8
I6	5/3 = 1.67

Step2: Sort Item (Descending Order of Ratio)

W = 16. Our
efit (value or
ciated weight

i5 (3)
i6 (1.67)
i4 (1.6)
i1 (1)
i3 (0.33)
i2 (0.2)

Step 3: Fill Knapsack (Greedy Method)

Capacity W = 16

Take i5 → Weight = 1, Value = 3
Remaining = 15

Take i6 → Weight = 3, Value = 5
Remaining = 12

Take i4 → Weight = 5, Value = 8
Remaining = 7

Take i1 → Weight = 6, Value = 6
Remaining = 1

o)

Take fraction of I3

Weight used = 1/3 of I3

Value = 1 x (1/3) = 0.33

Remaining = 0

Step4: Total Value = 8+5+8+6+0.33

= 22.33 (approx)

Final Answer

Maximum Profit ≈ 22.33

Conclusion

- This is solved using Greedy Method
- Item selected based on max Value/Weight ratio.
- Time Complexity = O(n log n) (due to sorting)

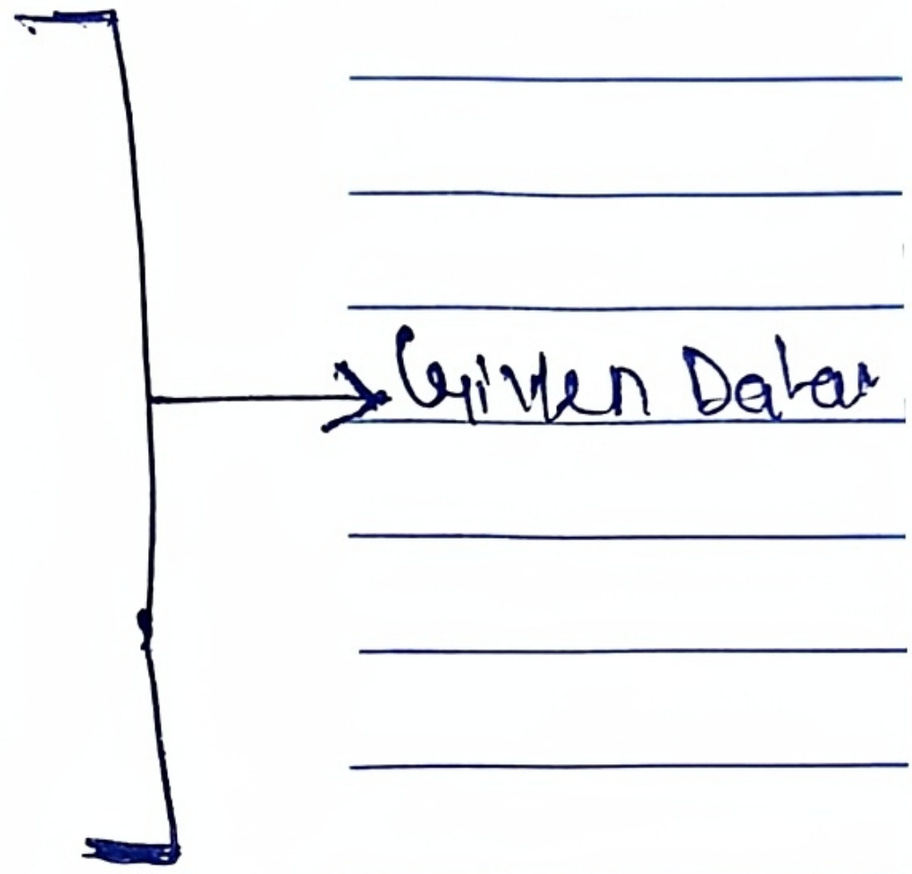
Step1

Step2

4. A st
crea
Cre
A
B
C
D
E
F
G
• D
• L
•

4. A studio is rented by different content creators *for recording videos. Only one* creator can use the studio at a time.

Creator	Start Time	End Time
A	2	3
B	1	4
C	3	5
D	0	6
E	5	7
F	8	9
G	5	9



- Determine the **maximum number of creators** that can record videos.
- List the selected activities. Illustrate the process using a timeline diagram.
- What is the time complexity of the greedy solution after sorting activities?

Step 1: Sort by End time.

A (2,3) ⟶ (2,3)
B (1,4) ⟶ (1,4)
C (3,5) ⟶ (3,5)
D (0,6) ⟶ (0,6)
E (5,7) ⟶ (5,7)
F (8,9) ⟶ (8,9)
G (5,9) ⟶ (5,9)

Step 2: Select Activities (Greedy Approach)

Select A (2,3) → first activity

Next:

C (3,5) → start ≥ 3 → select

Next:

E (5,7) → start ≥ 5 → select

Next:

F (8,9) → start ≥ 7 → select

Selected Activities

A → C → E → F

Maximum Number of Creators

Total = 4

Time Comp

Sorting =
Selection

Total = [

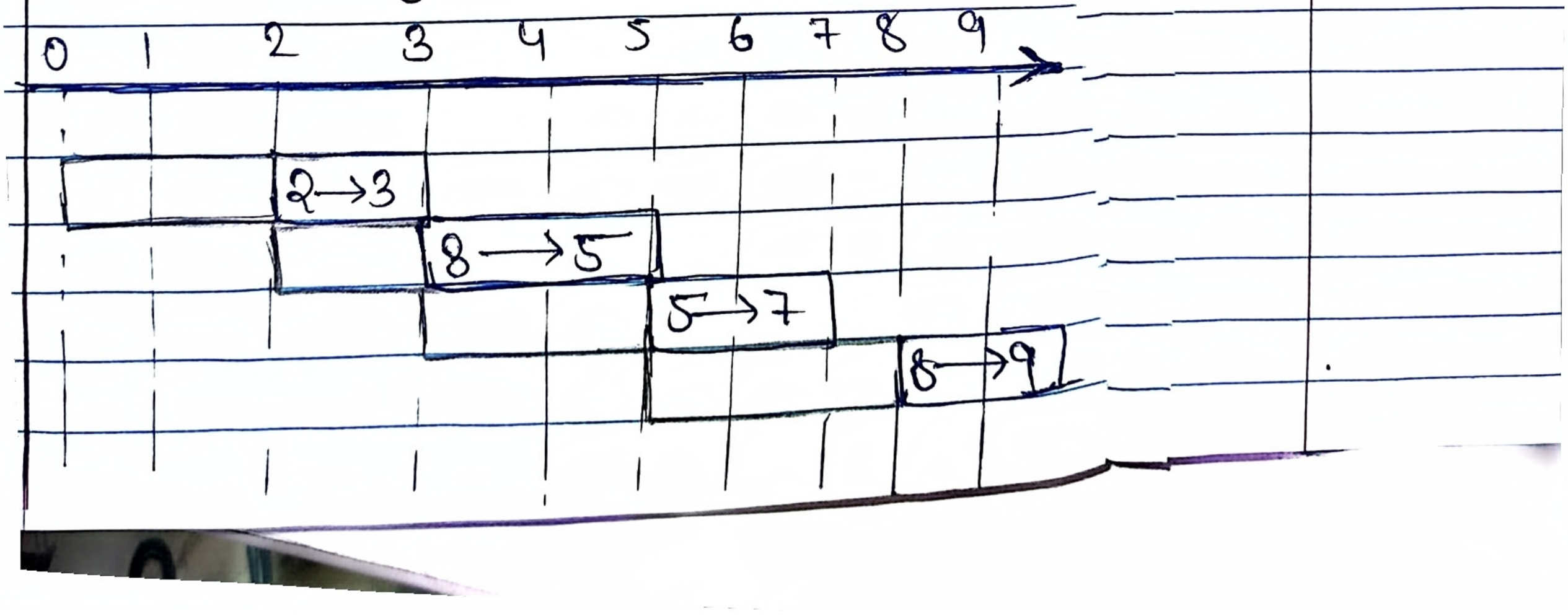
Conclusion

→ This is

→ Alway pic
time.

→ Maximum

Timeline Diagram



Time Complexity

Sorting = $O(n \log n)$
Selection = $O(n)$

Total = $\boxed{O(n \log n)}$

Conclusion

→ This is solved using Greedy Algorithm
→ Alway pick activity with earliest finish time.
→ Maximum non-overlapping activities = 4

5.	Compare **Greedy** and **Dynamic Programming** techniques with respect to: • Optimality • Overlapping subproblems • Time complexity

Basis	Greedy Algorithm	Dynamic Program.
• Optimality	Does not always give optimal solution.	Alway gives optimal solution.
• Overlapping Subproblems	Does not solve overlapping Subproblem	Solves overlapping Subproblems by storing result.
• Time Complexity	Usually faster.	usually slower than Greedy.
• Approach	Makes local optimal choice.	Solves all Subproblems and choose best.
• Memory Usage	less memory	More memory (uses table)
• Example Problems	Kruskal, Prim, Dijkstra, fractional Knapsack.	0/1 Knapsack, LCS, fibonacci, TSP.

Step 1: LC

If

If

Step 2:

	Y	0
	0	0
X	0	0
1	1	0
0	0	1
1	1	1
0	1	1
1	1	1
1	1	1

6. Determine the ***Longest Common Subsequence*** **(LCS)** of the sequences: {1 0 0 1 0 1 0 1} and {0 1 0 1 1 0 1 1 0}

Show the DP table and result.

Program

Sequences:

$X = \{1\,0\,0\,1\,0\,1\,0\,1\}$

$Y = \{0\,1\,0\,1\,1\,0\,1\,1\,0\}$

ves

Solution. Step 1: LCS formula (Dynamic Programming)

erlapping

blems

g result.

Repeat

If character match:

$L[i][j] = L[i-1][j-1] + L$

If character do not match:

$L[i][j] = \max(L[i-1][j], L[i][j-1])$

slower

greedy.

Step 2: DP Table.

all

blems

ose

		Y	0	0	1	0	1	1	0	1	1	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0
X	0	0	0	0	0	0	1	1	1	1	1	1	1
	1	1	0	1	1	1	1	2	2	2	2	2	2
	0	0	1	1	2	2	2	2	2	2	2	2	3
memory	1	1	1	2	2	3	3	3	3	3	3	3	3
table)	0	1	1	2	3	3	3	4	4	4	4	4	4
	1	1	1	2	3	4	4	4	5	5	5	5	6
napsack,	1	1	1	2	3	4	4	5	5	5	5	6	6
bonacci,													

Directions

Step 3: LCS Result
longest Common Subsequence = 100110
length of LCS = 6

Step 4: Time and space Complexity

[Time Complexity = O(m x n)
Space Complexity = O(m x n)]

Final Answer

The longest Common Subsequence of the sequences
{10010101} and {01011011} is:
LCS = 100110 length = 6

7.

Step 1:

Step 2:

7.	Obtain the solution to knapsack problem by Dynamic Programming *method* n=6, (p1, p2,…p6)=(w1,w2,…w6)=(100,50,20,10,7,3) and m=165

Given P_1 P_2 P_3 P_4 P_5 P_6

n=6. Profit = 100, 50, 20, 10, 7, 3
Weight = 100, 50, 20, 10, 7, 3
W_1 W_2 W_3 W_4 W_5 W_6

Capacity (m) = 165.

Step 1: Knapsack formula (DP)

If weight ≤ Capacity:

$$V[i][W] = \max(V[i-1][W], Pi + V[i-1][W-Wi])$$

Else:

$$V[i][W] = V[i-1][W]$$

Step 2: Observation

Here profit = weight, so we just try to fill bag as close to 165 possible.

Weights available: 100, 50, 20, 10, 7, 3

We need Sum = 165

Try combinations:

100+50+10+5 → not Possible.

100 + 50 + 7 + 3 = 160

100 + 20 + 10 + 7 + 3 = 140.

50 + 20 + 10 + 7 + 3 = 90

100 + 50 + 20 = 170 (exceeds)

100 + 50 + 10 = 160

100 + 50 + 7 + 3 = 160

100 + 20 + 10 + 7 + 3 = 140

50 + 20 + 10 + 7 + 3 = 90

Best Possible = 160

Step 3: Table Calculation (Final Result Table)

Item	Weight	Profit
1	100	100
2	50	50
3	20	20
4	10	10
5	7	7
6	3	3

Since total weight = 100 + 50 + 20 + 10 + 7 + 3

= 190 > 165, we Cannot take all item.

Steps.

Step 6:

Q8.

Step 5. Select item = {100, 50, 10, 3}
Total weight = 163
Total Profit = 163

Step 6: Time and Complexity = O(n x m)
Space and Complexity = O(n x m)

Q 8. Given a chai of some matrices A1, A2, A3, A4 with P0 = 5, P1 = 4, P2 = 6, P3 = 2 and P4 = 7 find m[1, 4].

Given:
Matrices $A_1, A_2 A_3, A_4$.
Dimensions:
$P_0 = 5$ $P_1 = 4$, $P_2 = 6$, $P_3 = 2$ and $P_4 = 7$

$A_1 = 5 \times 4$
$A_2 = 4 \times 6$
$A_3 = 6 \times 2$
$A_4 = 2 \times 7$

Step 1: Cost of multiplying two matrices

formula.

$$Cost = P(i-1) \times P(k) \times P(j)$$

Step 2 Calculate $m[i,j]$

length = 2

$m[1,2] = 5 \times 4 \times 6 = 120$

$m[2,3] = 4 \times 6 \times 2 = 48$

$m[3,4] = 6 \times 2 \times 7 = 84$

length = 3

$m[1,3] = \min$ {

$(m[1,1] + m[2,3] + 5 \times 4 \times 2)$

$= 0 + 48 + 40 = 88$

$(m[1,2] + m[3,3] + 5 \times 6 \times 2)$

$= 120 + 0 + 60 = 180$

}

$m[1,3] = 88$

$m[2,4] = \min$ {

$(m[2,2] + m[3,4] + 4 \times 6 \times 7)$

$= 0 + 84 + 168 = 252$

matrices

$(m[2,3] + m[4,4] + 4\times2\times7)$
$= 48 + 0 + 56 = 104$

$m[2,4] = 104$

length = 4

$m[1,4] = \min\{$
$(m[1,1] + m[2,4] + 5\times4\times7) = 0+104+140 = 244$
$(m[1,2] + m[3,4] + 5\times6\times7) = 120+84+210 = 414$
$(m[1,3] + m[4,4] + 5\times2\times7) = 88+0+70 = 158.$
$\}$

$m[1,4] = 158.$

Final DP Table.

i | j	1	2	3	4
1	0	120	88	158
2	0	0	48	104
3	0	0	0	84
4	0	0	0	0

Time Complexity

$O(n^3)$